claude-windows / e83b09d8-e113-4d4d-9411-cd80ae217af9

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf_ae339437-da9\agent-ae7785f29136202de.jsonl`
- SHA-256: `ed6971830a183fc8ef4e449f83c09601b18ded54e39243011ec3f109e25871ec`
- Source modified: `2026-07-07T15:53:47+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_ae339437-da9`
- session_id: `e83b09d8-e113-4d4d-9411-cd80ae217af9`

Transcript

1. user (2026-07-07T15:52:13.633Z)

```
You are reviewing ONE commit on branch `fix/cloud-run-exec-status-fastfail` in the repo at:
  C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
Always start by: cd "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer" (the
Bash cwd may default elsewhere). Use Read/Grep with absolute paths under that repo.

The commit hardens `CloudRunCompute.run_infer` (backend/compute/cloud_run.py): the done-marker
stays the
SOLE success signal, but each bucket-poll ALSO probes the Cloud Run Executions API so a worker
that
TERMINATED without ever writing a marker (crash/OOM/an old image argparse-aborting on an unknown
flag)
fails fast (raise RemoteExecutionFailed) instead of polling the full remote_poll_max_wait_sec
(~65 min).
`_dispatch_remote` maps RemoteExecutionFailed -> rc 5 (distinct from the rc 4 poll-timeout).

CRITICAL CONTEXT: this commit was REBASED onto a master that had since merged two sibling PRs
touching
the same files:
  - #513 (23c1868): added `ComputeJobSpec.skip_validation` + a `--skip-validation` worker flag;
the
    control-plane dispatcher sets it so the worker skips redundant re-validation.
  - #512 (20862d7): a validators shared-decode refactor (file-disjoint from this commit).
The motivating scenario for THIS fix: a post-#513 control-plane dispatching `--skip-validation`
to an
OLD worker image (no such flag) -> the worker argparse-aborts (exit 2) BEFORE writing a marker
-> the
exact ~65-min hang this fix collapses to a prompt rc-5 failure.

To see EXACTLY what this commit changes (only 5 files), run:
  git diff origin/master...HEAD        (three-dot: my change vs merge-base)
To see the sibling context that landed on the same files, run:
  git diff 3be0935..origin/master -- backend/compute/base.py backend/compute/cloud_run.py
backend/worker/__main__.py

Read the FULL current files (not just the diff) to judge correctness in context:
  backend/compute/cloud_run.py, backend/compute/base.py, backend/compute/__init__.py,
  backend/worker/__main__.py, backend/orchestration.py (the API caller),
tests/test_compute_backend.py,
  backend/infrastructure/execution_policy.py (poll_until semantics).

Report ONLY real, defect-grade findings with a concrete failure scenario. Do NOT report style
nits,
things the gate already covers, or speculation. If the change is correct on your lens, return an
```

empty
findings array and say so in `assessment`. Full suite (1833) + ruff + mypy + coverage 85.66%
already pass;
do not re-report "add a test" unless a SPECIFIC untested behavior would plausibly break.

Adversarially VERIFY this finding by reading the ACTUAL code. Default to REFUTED unless the code
truly exhibits
the defect. A finding is CONFIRMED only if you can trace a concrete input/state to the claimed
wrong behavior.

FINDING (realclient / blocker / correctness) at backend/compute/cloud_run.py:190:
GoogleCloudRunJobClient.execution_terminal_state ALWAYS returns None on the real client, so the
fast-fail path is completely inert in production and the ~65-min hang this commit exists to
collapse is not actually fixed.
The terminal-detection guard reads `completion_time.seconds`:

```
    if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
        return None
```

The comment asserts "completion_time is a proto Timestamp", but google-cloud-run (run_v2) is a
proto-plus library, and proto-plus marshals `google.protobuf.Timestamp` fields to native Python
datetimes, NOT to a proto Timestamp with a `.seconds` field. I verified this against the
installed `google-cloud-run` / proto-plus 1.28.0:
  - UNSET completion_time (still running) -> exe.completion_time is None
  - SET   completion_time (terminal)   -> exe.completion_time is a
proto.datetime_helpers.DatetimeWithNanoseconds (a datetime.datetime subclass)
A datetime has `.second` (singular), never `.seconds`. So `getattr(<datetime>, "seconds", 0)` is
always 0, `not 0` is always True, and the function returns None on line 190 for EVERY execution
-- including a definitively-terminal, completion_time-set, failed_count=1 one. I reproduced the
real classifier body end-to-end: classify(terminal-Failed execution) => None (expected
'Failed'). The cancelled/failed/succeeded classification below (lines 193-199) is therefore
unreachable dead code on the real path.

Every unit test passes only because the fakes (_CrashedNoMarkerClient, _MarkerAndFailedStatus,
etc.) OVERRIDE execution_terminal_state to return a plain string, so they never exercise the
real proto-parsing method (which is pragma:no-cover). The seam looks wired but the sole real
implementation never signals a terminal state.

Net effect: on the only path that matters (production Cloud Run), _terminal_execution_state
always sees None, _poll_check always returns None, and run_infer degrades to exactly the pre-
commit marker-only polling -- i.e. the fix is a no-op in prod while giving green-test false
confidence.

Fix: distinguish running vs terminal via the marshaled datetime itself, e.g. `if getattr(exe,
"completion_time", None) is None: return None` (proto-plus returns None for an unset Timestamp
and a real datetime once terminal). Be defensive about a possible epoch-datetime for unset in
other proto-plus versions.
Claimed failure scenario: The exact motivating scenario: post-#513 control-plane dispatches
`--skip-validation` to an OLD worker image with no such flag. The worker argparse-aborts (exit
2) BEFORE writing any done-marker. The Cloud Run execution reaches terminal Failed
(completion_time set, failed_count=1). On the next bucket-poll, _poll_check ->
_terminal_execution_state -> real execution_terminal_state calls get_execution, but the
completion_time.seconds guard evaluates `not getattr(<DatetimeWithNanoseconds>, 'seconds', 0)`
== `not 0` == True and returns None. _poll_check returns None and keeps polling.
RemoteExecutionFailed/rc-5 NEVER fires; the run hangs the full remote_poll_max_wait_sec (~65
min) then raises PolicyTimeout (rc 4) -- precisely the opaque long hang this commit claims to
eliminate.

2. user (2026-07-07T15:52:19.258Z)

145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)

66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
---BRANCH---
fix/cloud-run-exec-status-fastfail

3. user (2026-07-07T15:52:19.266Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14           *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15           Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16           worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17           (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26      owner-verified on the M7 real run.
27      """
28
29      from __future__ import annotations
30
31      import json
32      import logging
33      import os
34      import tempfile
35      import uuid
36      from abc import ABC, abstractmethod
37      from typing import Any, Callable, List, Optional
38
39      from backend.compute.base import (
40          ComputeBackend,
41          ComputeJobSpec,
```

```
        ComputeResult,
        RemoteExecutionFailed,
    )
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
    )
    from backend.storage import fetch, get_storage, parse_uri

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""

        @abstractmethod
        def cancel_execution(self, execution: str) -> None:
            """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
            times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
            May raise ? the caller treats every failure as best-effort (the original poll
timeout is
            NEVER masked by a cancel error)."""

        def execution_terminal_state(self, execution: str) -> Optional[str]:
            """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
            ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
            can't be determined).

            ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
            terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
```

```
fast instead
95              of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97              NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98              chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99              polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104     class GoogleCloudRunJobClient(CloudRunJobClient):
105         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108         def run_job(
109             self,
110             job_name: str,
111             argv: List[str],
112             *,
113             region: str,
114             project: Optional[str] = None,
115         ) -> str:
116             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119             if not project:
120                 raise RuntimeError(
121                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123                 )
124             try:
125                 from google.cloud import run_v2  # type: ignore[attr-defined]
126             except Exception as exc:  # pragma: no cover - real SDK not present in CI
127                 raise RuntimeError(
128                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                     "control plane (it is intentionally NOT a CI/test dependency)."
130                 ) from exc
131             client = (
132                 run_v2.JobsClient()
133             )  # pragma: no cover - exercised only on the real M7 run
134             name = client.job_path(project, region, job_name)
135             overrides = run_v2.RunJobRequest.Overrides(
136                 container_overrides=[
137                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
138                 ]
139             )
140             op = client.run_job(
141                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
142             )
143             return (
144                 getattr(op, "metadata", None)
145                 and getattr(op.metadata, "name", "")
```

```
146                 or "execution"
147             )
148
149         def cancel_execution(self, execution: str) -> None:
150             # run_job falls back to the literal string "execution" when the operation
metadata carries
151             # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
152             # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
153             # google-cloud-run is absent).
154             if "/executions/" not in (execution or ""):
155                 raise RuntimeError(
156                     f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
157                     "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
158                     "one from the operation metadata, so there is nothing to cancel by
name)."
159                 )
160             try:
161                 from google.cloud import run_v2  # type: ignore[attr-defined]
162             except Exception as exc:  # pragma: no cover - real SDK not present in CI
163                 raise RuntimeError(
164                     "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
165                     "control plane (it is intentionally NOT a CI/test dependency)."
166                 ) from exc
167             client = (
168                 run_v2.ExecutionsClient()
169             )  # pragma: no cover - exercised only on a real run
170             client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172         def execution_terminal_state(self, execution: str) -> Optional[str]:
173             # Advisory status probe (see the ABC docstring): describe the execution and,
ONLY once it
174             # is terminal (``completion_time`` set), classify it. A non-resolvable name /
absent SDK
175             # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
never
176             # raises on bad input, because the probe is best-effort and must not fail a
healthy run.
177             if "/executions/" not in (execution or ""):
178                 return None
179             try:
180                 from google.cloud import run_v2  # type: ignore[attr-defined]
181             except Exception:  # pragma: no cover - real SDK not present in CI
182                 return None
183             client = (
184                 run_v2.ExecutionsClient()
185             )  # pragma: no cover - exercised only on a real run
186             exe = client.get_execution(  # pragma: no cover - real run only
187                 request=run_v2.GetExecutionRequest(name=execution)
188             )
189             # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
190             if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
191                 return None  # pragma: no cover - real run only
192             # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
193             if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
194                 return "Cancelled"
195             if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
196                 return "Failed"
197             if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
```

```
                return "Succeeded"
            return "Failed"  # pragma: no cover - terminal but no positive success ? treat
as failed


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            if spec.skip_validation:
                # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
                # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
                argv.append("--skip-validation")
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]
```

```
258
259             execution = self._client.run_job(
260                 self._job_name, argv, region=self._region, project=self._project
261             )
262             LOG.info(
263                 "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
264                 self._job_name,
265                 execution,
266                 done_uri,
267             )
268
269             try:
270                 marker = poll_until(
271                     lambda: self._poll_check(done_uri, str(execution or "")),
272                     self._policy,
273                     label="cloud-run-infer",
274                     logger=LOG,
275                     # Live heartbeat (#482): each still-waiting tick reaches the caller so
276                     # job.progress moves during the whole GPU run instead of freezing.
277                     on_tick=on_wait,
278                 )
279             except PolicyTimeout as timeout_exc:
280                 # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
281                 # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
282                 # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
283                 marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
284             video_id = str(marker.get("video_id", ""))
285             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
286             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
287             rc = int(marker.get("returncode", 1))
288             if not video_id:
289                 LOG.warning(
290                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
291                     "video_id) ? treating as failure",
292                     self._job_name,
293                 )
294                 rc = rc or 1
295             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
296             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
297             LOG.info(
298                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
299             )
300             return ComputeResult(
301                 returncode=rc,
302                 outputs_uri=outputs_uri,
303                 video_id=video_id,
304                 report=report,
305                 execution=str(execution or ""),
306             )
307
308         def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
309             """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
310
311             The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
312             wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
313             FAILURE* marker resolves through the normal path with its real returncode. Only
when the
```

```
            marker is absent do we consult the execution status: a worker that TERMINATED
without ever
            writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
            otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
            opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
            failure.

            Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
            ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
            will ever appear)."""
            marker = self._read_marker(done_uri)
            if marker is not None:
                return marker  # success signal ? authoritative even if status later flips
            state = self._terminal_execution_state(execution)
            if state is None:
                return None  # still pending/running, or status unknown ? keep bucket-
polling
            # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
            # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
            # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
            # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
            marker = self._read_marker(done_uri)
            if marker is not None:
                return marker
            raise RemoteExecutionFailed(
                f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
                f"terminal state {state!r} with NO completion marker ? the worker exited
before "
                "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
                "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
                "inspect the Cloud Run Job execution log to diagnose: "
                f"`gcloud run jobs executions describe {execution or '<execution>'} "
                f"--region {self._region}`."
            )

        def _terminal_execution_state(self, execution: str) -> Optional[str]:
            """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
            flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
            so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
            ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
            if not execution:
                return None
            try:
                return self._client.execution_terminal_state(execution)
            except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
                LOG.debug(
                    "cloud-run: execution status probe failed for %s ? treating as unknown",
                    execution,
                    exc_info=True,
                )
```

```
                return None

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
                )
                return late
            try:
                self._client.cancel_execution(execution)
            except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
                cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
                LOG.warning(
                    "cloud-run: best-effort cancel of execution=%s failed: %s",
                    execution,
                    cancel_exc,
                    exc_info=True,
                )
            else:
                cancel_note = "cancel requested"
                LOG.info("cloud-run: requested cancel of execution=%s", execution)
            raise PolicyTimeout(
                f"{timeout_exc} [cloud-run job={self._job_name} "
                f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
                "state in the GCP console]"
            ) from timeout_exc

        # --- storage-backed completion signal (bucket-poll) ---
        def _read_marker(self, done_uri: str) -> Optional[dict]:
            """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
            ref = parse_uri(done_uri)
```

```
414             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
415                 return None
416             return (
417                 self._read_json(done_uri) or {}
418             )  # empty dict still terminates the poll (present == done)
419
420         def _read_json(self, uri: str) -> dict:
421             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
422             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
423             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
424             try:
425                 ref = parse_uri(uri)
426                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
427                     dst = os.path.join(td, ref.name or "obj.json")
428                     local = fetch(uri, dst, config=self._storage_config)
429                     with open(local, "r", encoding="utf-8") as f:
430                         data: Any = json.load(f)
431                 return data if isinstance(data, dict) else {}
432             except Exception:
433                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
434                 return {}
435
```

4. user (2026-07-07T15:52:28.336Z)

```
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2\__init__.py
---PROTOPLUS---
proto-plus 1.28.0
```

5. user (2026-07-07T15:52:40.330Z)

```
=== UNSET completion_time ===
type: <class 'NoneType'>
repr: None
getattr(exe, completion_time, None) is None -> True
getattr(ct, seconds, 0) -> 0
bool(guard not ...) -> True

=== SET completion_time (terminal) ===
type: <class 'proto.datetime_helpers.DatetimeWithNanoseconds'>
repr: DatetimeWithNanoseconds(2026, 7, 7, 12, 0, tzinfo=datetime.timezone.utc)
has .seconds attr -> False
has .second attr -> True
getattr(ct2, seconds, 0) -> 0
GUARD not getattr(...seconds,0) -> returns None?: True
ct2 is None -> False
```

6. user (2026-07-07T15:52:56.379Z)

```
classify(terminal-Failed) -> None (expected: Failed)
classify(terminal-Cancelled) -> None (expected: Cancelled)
classify(terminal-Succeeded) -> None (expected: Succeeded)
classify(running) -> None (expected: None)
```

7. assistant (2026-07-07T15:53:02.755Z)

```
Confirmed the core mechanism. Let me verify the test-fakes claim and that no test exercises the
real proto-parsing path.
```

8. user (2026-07-07T15:53:05.083Z)

```
140:    def execution_terminal_state(self, execution):
176:    """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is unset
?
180:    from backend.compute.cloud_run import GoogleCloudRunJobClient
182:    client = GoogleCloudRunJobClient()
195:    from backend.compute.cloud_run import GoogleCloudRunJobClient
197:    client = GoogleCloudRunJobClient()
547:        def execution_terminal_state(self, execution):  # pragma: no cover - never reached
567:        def execution_terminal_state(self, execution):
585:def test_default_execution_terminal_state_is_none():
589:    assert _NoopRunClient().execution_terminal_state("whatever") is None
591:        _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
598:    from backend.compute.cloud_run import GoogleCloudRunJobClient
600:    client = GoogleCloudRunJobClient()
602:        assert client.execution_terminal_state(bogus) is None
646:        def execution_terminal_state(self, execution):  # pragma: no cover - must never be
called
```

9. user (2026-07-07T15:53:08.080Z)

```
130
131         def run_job(self, job_name, argv, *, region, project=None):
132             self.calls.append(
133                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
134             )
135             return self.EXEC_NAME
136
137         def cancel_execution(self, execution):
138             self.cancelled.append(execution)
139
140         def execution_terminal_state(self, execution):
141             self.status_polls += 1
142             # Anti-hang guard: the fast path MUST abort on the first terminal observation.
If a
143             # regression kept polling despite a terminal status, trip here instead of
spinning.
144             assert self.status_polls <= 5, "status probed repeatedly ? fast path did not
abort the poll"
145             return self._state
146
147
148     # ------------------------------------------------------------------ factory (default-
OFF)
149
150
151     def _cfg(compute_target="", storage_backend="local"):
152         return AppConfig.model_validate(
153             {
154                 "deployment": {"compute_target": compute_target},
155                 "storage": {"backend": storage_backend},
156             }
157         )
158
159
160     def test_factory_is_none_for_local_targets():
161         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
162         assert get_compute_backend(_cfg("")) is None
163         assert get_compute_backend(_cfg("local_gpu")) is None
164         assert get_compute_backend(_cfg("cpu_mock")) is None
165
166
167     def test_factory_returns_cloud_run_for_cloud_target():
168         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
169         assert isinstance(backend, CloudRunCompute)
```

```
170
171
172     # ------------------------------------------------------ real client: missing-project
guard
173
174
175     def test_real_client_rejects_missing_project():
176         """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is
unset ?
177         a None/empty project would build a bogus "projects/None/..." job path (job_path
requires a
178         real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
179         (the SDK is intentionally not a test dependency)."""
180         from backend.compute.cloud_run import GoogleCloudRunJobClient
181
182         client = GoogleCloudRunJobClient()
183         for missing in (None, ""):
184             with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):
185                 client.run_job(
186                     "rally-worker", ["infer"], region="asia-south1", project=missing
187                 )
188
189
190     def test_real_client_cancel_rejects_unresolvable_execution_name():
191         """cancel_execution must fail loudly on a non-resolvable execution name ? run_job
falls back
192         to the literal "execution" when the operation metadata carries no name, and
cancelling that
193         would be a bogus API call. The guard runs BEFORE the google-cloud-run import
(mirrors the
194         project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
195         from backend.compute.cloud_run import GoogleCloudRunJobClient
196
197         client = GoogleCloudRunJobClient()
198         for bogus in ("", "execution", "exec-123"):
199             with pytest.raises(RuntimeError, match="fully-qualified"):
200                 client.cancel_execution(bogus)
201
202
203     # ------------------------------------------------------------ CloudRunCompute dispatch
+ poll
204
```

10. user (2026-07-07T15:53:08.943Z)

```
585     def test_default_execution_terminal_state_is_none():
586         """The ABC default is None (additive / default-OFF): a client that doesn't override
status
587         polling degrades transparently to marker-only behaviour, so every pre-existing fake
is
588         unaffected and the seam never changes a healthy run's outcome."""
589         assert _NoopRunClient().execution_terminal_state("whatever") is None
590         assert (
591             _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
592         )
593
594
595     def test_real_client_terminal_state_none_for_unresolvable_name():
596         """The real client's status probe is best-effort: a non-resolvable execution name
returns None
597         (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
absent)."""
598         from backend.compute.cloud_run import GoogleCloudRunJobClient
```

```
599
600         client = GoogleCloudRunJobClient()
601         for bogus in ("", "execution", "exec-123"):
602             assert client.execution_terminal_state(bogus) is None
603
604
```

11. assistant (2026-07-07T15:53:39.375Z)

The finding is fully confirmed by direct reproduction against the installed libraries. Let me record the verdict.

Evidence gathered:

1. **Library reality**: `google-cloud-run` (`run_v2`) is installed and `proto-plus 1.28.0` is the marshaling layer. `run_v2.Execution` is a proto-plus message.

2. **Marshaling behavior reproduced** on a real `run_v2.Execution`:
   - UNSET `completion_time` ? `exe.completion_time is None`
   - SET `completion_time` ? `proto.datetime_helpers.DatetimeWithNanoseconds` (a `datetime.datetime` subclass). It has `.second` (singular), and `hasattr(ct, "seconds")` is **False**.

3. **The guard is inert**: `not getattr(getattr(exe, "completion_time", None), "seconds", 0)` evaluates to `not 0 == True` in *both* the unset case (`getattr(None, "seconds", 0)==0`) and the terminal case (`getattr(<datetime>, "seconds", 0)==0`). So line 190-191 returns `None` for every execution.

4. **End-to-end classifier reproduction** (exact copy of lines 189-199): `classify(terminal-Failed, failed_count=1) -> None` (expected `"Failed"`); Cancelled ? None; Succeeded ? None. The classification block on lines 193-199 is unreachable dead code on the real path.

5. **Tests never catch it**: `test_real_client_terminal_state_none_for_unresolvable_name` only exercises the early `"/executions/" not in name` return (line 177-178); the proto-parsing body is `# pragma: no cover - real run only`. Every `CloudRunCompute` test uses fakes (`_FakeRunClient`, `_CrashedNoMarkerClient`, etc.) that override `execution_terminal_state` to return a plain string, so the real `.seconds` guard is never run.

Net effect matches the claim: in production, `_terminal_execution_state` always sees `None`, `_poll_check` always returns `None`, and `run_infer` degrades to pre-commit marker-only polling. The motivating scenario (old worker argparse-aborts on `--skip-validation`, terminal Failed, no marker) still hangs the full `remote_poll_max_wait_sec` (~65 min) ? `PolicyTimeout` (rc 4); `RemoteExecutionFailed` (rc 5) never fires. The fix is a no-op in prod with green-test false confidence.

The suggested fix (`if getattr(exe, "completion_time", None) is None: return None`) is validated by my reproduction: unset?None (keeps polling), set?non-None datetime (proceeds to classify).

12. user (2026-07-07T15:53:47.276Z)

Structured output provided successfully